

A Survey on Performance Modeling and Prediction for Distributed DNN Training

Zhenhua Guo, Yanan Tang , Jidong Zhai , Senior Member, IEEE, Tongtong Yuan , Jian Jin , Li Wang, Yaqian Zhao, and Rengang Li 

Abstract—The recent breakthroughs in large-scale DNN attract significant attention from both academia and industry toward distributed DNN training techniques. Due to the time-consuming and expensive execution process of large-scale distributed DNN training, it is crucial to model and predict the performance of distributed DNN training before its actual deployment, in order to optimize the design of distributed DNN training at low cost. This paper analyzes and emphasizes the importance of modeling and predicting the performance of distributed DNN training, categorizes and analyses the related state-of-the-art works, and discusses future challenges and opportunities for this research field. The objectives of this paper are twofold: first, to assist researchers in understanding and choosing suitable modeling and prediction tools for large-scale distributed DNN training, and second, to encourage researchers to propose more valuable research about performance modeling and prediction for distributed DNN training in the future.

Index Terms—Distributed DNN training, distributed computing systems, large-scale machine learning, performance modeling.

I. INTRODUCTION

DEEP Neural Network (DNN) models, especially deep learning models with large numbers of parameters, have made significant breakthroughs in recent years [1], [2]. For example, in the field of natural language processing, several large language models such as ChatGPT [3] and LLaMA [4] are capable of not only understanding human language but also engaging in generating natural conversational interactions. In the field of computer vision, SAM [5] has made remarkable advancements in image segmentation tasks, by enabling the segmentation of any object in any image without using any annotations.

The development of large-scale DNN models heavily relies on the exponential growth of their model parameter size.

Received 27 October 2023; revised 12 September 2024; accepted 28 September 2024. Date of publication 7 October 2024; date of current version 25 October 2024. This work was supported in part by National Key Research and Development Program of China under Grant 2021ZD0113000. This paper was produced by IEIT SYSTEMS Co., Ltd. It is in Jinan, China. Recommended for acceptance by E. Smirni. (Corresponding author: Yanan Tang.)

Zhenhua Guo, Yanan Tang, Li Wang, Yaqian Zhao, and Rengang Li are with the IEIT SYSTEMS Company, Ltd., Jinan 250101, China (e-mail: guozhenhua@ieisystem.com; tangyanan@ieisystem.com; wang_libj@ieisystem.com; zhaoyaqian@ieisystem.com; lrg@ieisystem.com).

Jidong Zhai is with the Tsinghua University, Beijing 100084, China (e-mail: zhajidong@gmail.com).

Tongtong Yuan is with the Beijing University of Technology, Beijing 100124, China (e-mail: yuantt@bjut.edu.cn).

Jian Jin is with the CAICT, Beijing 100191, China (e-mail: jin.jian@caict.cn). Digital Object Identifier 10.1109/TPDS.2024.3476390

Nowadays, large DNN models such as GPT-3 [6] have reached parameter sizes as high as 175 billion, which makes it challenging to process model training on a single machine. To realize the model training in such scenarios, distributed DNN training has become an essential technology in the field of deep learning. Especially in recent years, applying hundreds or thousands of GPUs for distributed training has become a standard configuration in both industry and academia [7]. However, distributed training is time-consuming and expensive, while its performance is influenced by complex factors, such as computation power, parallel strategy, communication operators, computing clusters, etc. This makes it difficult to achieve optimal training performance by directly using deep learning frameworks such as TensorFlow [8], [9] and PyTorch [10]. Therefore, how to effectively optimize the performance of distributed training has become a key focus of Artificial Intelligence (AI) technology.

Nevertheless, the cost of actually deploying distributed training is often expensive, making it impractical to use a trial-and-error way to optimize distributed training in real systems. For example, renting 1,024 GPUs on Amazon costs approximately 3,548 dollars per hour [11], training the Gopher model [12] using distributed training consumes around 1,250 megawatt-hours [13], while training a GPT-3 model on AWS can cost over one million dollars in storage, computation, and energy consumption [14]. As a result, the best approach is to model and predict the performance of distributed DNN training, in order to explore the impact of various design spaces and search for the optimal strategy to deploy distributed training.

This paper presents a survey on the studies of performance modeling and prediction for distributed DNN training, by giving a classification for state-of-the-art studies and providing a brief description of their design principles and simulation capabilities. Besides, this paper also analyzes and emphasizes the motivation of performance modeling and prediction for distributed training, discusses the main design spaces that can be modeled, analyzes the application scenarios and selection of different categories of modeling and prediction works, and further presents the challenges and future directions. On one hand, this paper aims to help researchers understand and choose suitable modeling and prediction tools for optimizing their distributed training before actual deployment. On the other hand, this paper also encourages researchers to propose more valuable research on performance modeling and prediction for distributed DNN training, in order to assist the industry and academia in exploring new software, hardware, or deployment strategies for distributed training.

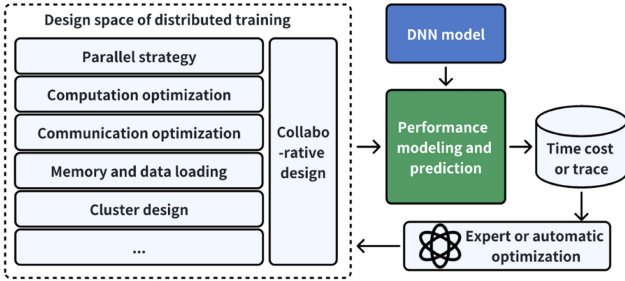


Fig. 1. Optimizing the design spaces of distributed training using performance modeling and prediction.

II. BACKGROUND AND MOTIVATIONS

A. Why Modeling and Predicting the Performance for Distributed DNN Training

Although distributed training can be automatically realized based on existing deep learning frameworks (e.g. Tensorflow and Pytorch), it remains challenging to ensure optimal execution [15]. Suppose that researchers want to optimize distributed training in a practical deployment system, it is necessary to make modifications to both the computing cluster and deep learning framework software, meanwhile developing a monitor. Furthermore, during the optimizing process, multiple iterations of distributed training are required to be executed on a large-scale cluster, which will inevitably incur significant computational resource usage, energy consumption, and expensive labor costs. By contrast, as depicted in Fig. 1, performance modeling and prediction provide an alternative approach where distributed training is virtually modeled instead of running on a real computing cluster, aiming to predict the time cost or the running trace of distributed training. By decoupling the optimization process from the underlying runtime platform through modeling and prediction, researchers can explore parallel strategies, computing cluster configurations, and even more design spaces of distributed training at low cost, while avoiding making complex and time-consuming modifications to the hardware and software.

Although there have been some survey articles about distributed training [16], [17], [18], they mainly focus on the deployment methods or training strategies, with limited coverage on the performance modeling and prediction studies. In order to fill this gap, this article provides a comprehensive survey on performance modeling and prediction for distributed DNN training.

B. Main Design Spaces Should Be Modeled

In this section, we briefly introduce the main design spaces of distributed training as exhibited in Fig. 1, which have a great impact on distributed DNN training.

1) *Parallel Strategies*: Parallel strategies are used to allocate computing tasks to GPUs (or computing nodes) in a computing cluster, aiming to enable multiple GPUs to perform efficient DNN training collaboratively. Common parallel strategies can be categorized into Data Parallelism (DP), Model Parallelism (MP), and Hybrid Parallelism (HP).

DP is primarily designed for distributed training scenarios with large training datasets. In DP, each computing node within a computing cluster maintains a replica of DNN and its own subset of training data. Each computing node independently performs the forward and backward propagation, and the gradients are then accumulated for model parameter synchronization. There are two common parallel modes for DP, including centralized mode and decentralized mode. In the centralized mode, the workers transmit their computed gradients to the Parameter Servers (PS) [19] and receive the most recent synchronized model state in return. In the decentralized mode, the parameter servers are removed, and weight gradients are exchanged among all nodes through all-reduce collective communications [20], [21]. The optional synchronization strategy of model parameters mainly includes Bulk Synchronous Parallel (BSP) [22], Staleness Synchronous Parallel (SSP) [23], and Asynchronous Parallel (ASP) [24]. Although SSP and ASP can accelerate distributed training to some extent, they would inevitably cause a decrease in the accuracy of DNN compared with BSP [25]. Therefore, BSP is currently the most widely accepted and used synchronization strategy in the industry.

In terms of MP, the DNN model is partitioned, aiming to address the issue that the DNN model is too large to be trained on a single computing node. This can be achieved by either partitioning the tensors (also called tensor MP) or performing multi-stage partitioning along the depth of DNN (also called pipeline MP). For tensor MP, the weight matrixes are divided and allocated to multiple computing nodes. Each computing node performs computations using its allocated part of the weight matrix with the collected input, and subsequently broadcasts its partial outputs to other computing nodes. For pipeline MP, the input data is further divided into micro-batches [26], and then sequentially inputted into a computing cluster with a pipeline fashion. Research such as Dapple [27] and Chimera [28] have further proposed various effective scheduling algorithms for pipeline MP. In addition, Expert Parallelism (EP) [29] designed for Mixture-of-Expert (MoE) models [30] can be regarded as a variant of MP.

By leveraging parallel strategies of DP and MP with different dimensions, HP is able to further optimize distributed training [31], [32]. Taking GPT-3 [6] as an example, its HP-based training strategy is shown in Fig. 2. To achieve efficient HP and reduce the difficulty for researchers in designing HP strategies, some approaches propose automated parallelism [33] to search for the optimal HP strategies [34], [35], [36], [37]. These methods typically rely on inserted modeling or prediction technology for distributed training, aiming to guide the optimization objectives of the HP strategy searching algorithm. For instance, Alpa [35] models the intra-operator execution cost and the inter-operator parallelization latency, then searches for the optimal HP strategy by minimizing both of them.

Different parallel strategies can be applied to adapt different training scenarios, yielding varying training acceleration effects. Therefore, designing an optimal distributed training parallel strategy has emerged as a central concern in the field of distributed training. This is also one of the key motivations for studying the performance modeling and prediction for

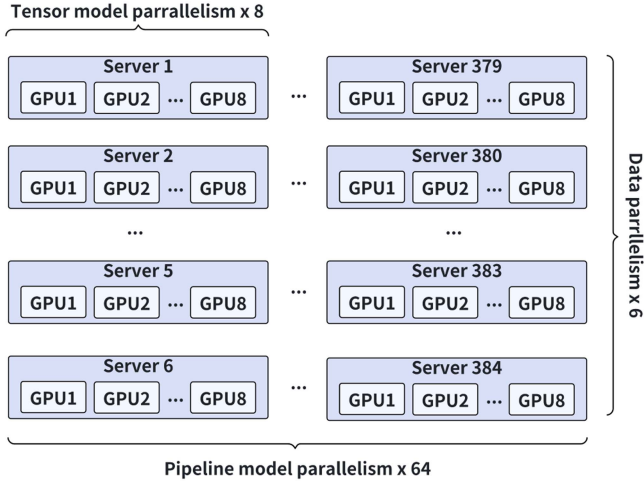


Fig. 2. The HP-based strategy of training GPT-3. During the training process, GPT-3 is first divided into 64 stages for pipeline MP. Each stage is executed on 6 servers, and within these servers, DP is performed. Furthermore, each server is equipped with 8 GPU cards, and tensor MP is applied among them.

distributed training, which can evaluate the parallel strategies without actual deployment.

2) *Computation Optimization*: Computation optimization refers to optimizing the computation processes, which offers extensive opportunities for exploration. On one hand, it is possible to optimize the allocation for the sub-computation tasks among the inter computing nodes [38]. On the other hand, for the computation process inside one computing node, several computation factors such as the training-precision settings during training [39] and the batch size used [40] can all impact the training performance. Moreover, the works for optimizing computing operators [41] and AI compilers [42] can also be seen as the computation optimizations at the software level. With the help of performance modeling and prediction for distributed training, design spaces of the computation optimization can be explored at a low cost, avoiding the time-consuming development and actual execution.

3) *Communication Optimization*: Numerous research works have demonstrated that communication has become a major performance bottleneck for distributed training [17]. To optimize the collective communication operators during distributed training such as all-reduce, reduce-scatter, and all-gather, several recent studies proposed to accelerate the operators themselves. For example, [43] proposed hierarchical all-reduce communication and tree-structured all-reduce communication, [44] presented topology-aware collective communication, [45] suggested compression techniques for reducing the communication volume of collective communication, and so on. Besides, optimizing the splitting and fusion strategies of collective communication operators (e.g. the tensor fusion in Horovod [20]) is also a promising approach [46]. As a result, communication optimization is an extensively studied design space, and it is also a focus of many performance modeling and prediction works for distributed DNN training.

4) *Memory and Data Loading*: On one hand, memory optimization enables GPU to train larger models with limited

memory, which can save a lot of costs and has become a key research focus in recent years. For example, ZeRO [47] proposes a data DP-based training approach that splits model parameters, gradients, and optimizer states among multiple GPUs, achieving significant memory savings with only 1.5 times additional communication overhead, and its subsequent work [48], [49] goes further in tapping into CPU memory and NVMe storage. Checkmate [50] employs recomputation techniques [51], [52] to trade off computation for additional memory space. Data encoding and compression are used in the work [53] to reduce memory consumption during training. In recent years, new protocols such as NVLink-C2C [54] and CXL [55] also fall under memory optimization by designing consistent multi-device memory interconnect protocols, which can help realize collaborative caching [56] to enhance the performance of distributed training, especially for DP that requires a large amount of memory.

On the other hand, the process of loading training data is also a key design space [57]. Designers need to consider how to effectively read training data from the storage (e.g., Hadoop Distribute File System [58]) to ensure that I/O does not limit the training speed. Besides, the training performance is usually affected by the data stall phenomenon in the data pipeline (including reading, preprocessing, shuffling training samples, and assembling mini-batches) [59], which involves the complicated design and optimization of both software and hardware [60].

In short, both memory optimization and data loading acceleration are challenging for actual deployment, so it is better to predict and quantify their effects using modeling and prediction technologies.

5) *Cluster Design*: As the underlying infrastructure for distributed training, the cluster design is one of the most critical design spaces that affects the training performance. However, designing and optimizing cluster design in real-world systems often involves time-consuming modifications and restructuring of cluster architecture, as well as hardware migrations and changes. In contrast, using performance modeling and prediction for distributed training allows researchers to explore the cluster design in a sandbox environment, aiding in exploring cluster design much more easily.

For the technology of cluster design, regarding the computing nodes, works such as DGX [61] have demonstrated the importance of designing specialized servers for distributed training. Besides, several studies have introduced new hardware architectures such as Field Programmable Gate Arrays (FPGA) to further optimize distributed training. For the design of interconnection between cluster nodes, numerous works focus on designing new cluster topologies. For example, industry clusters like Facebook's Zion platforms [62] and Google's TPU platforms [63], as well as academic studies such as [64], all try to optimize the collaborative efficiency between computing nodes at the topology level. Unlike traditional topologies such as tree-based data center architectures and 3D-torus architecture, cluster architectures are also promising to be extended to higher-dimensional (e.g. 4D/5D) interconnects to enhance the network capacity between computing nodes [65]. Moreover, the choice of network transmission protocols (such as RoCE, Infiniband, TCP/IP) also impacts the performance of distributed training.

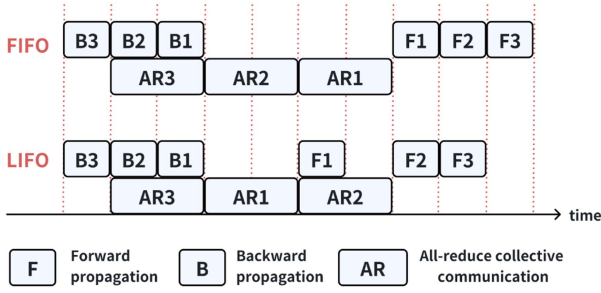


Fig. 3. This figure shows an example of realizing communication and computation optimization for training a 3-tensor DNN, which is trained based on DP. The numbers in the figure stand for the communication or the computation operated on the specific tensor. As can be observed from the example, compared with the First-In-First-Out (FIFO) order of collective communication, the Last-In-First-Out (LIFO) can accelerate the distributed training process by achieving better communication-computation overlap.

6) *Collaborative Design*: Different design spaces may also have mutual influences, which presents further challenges. For example, in some multidimensional cluster topologies such as 3D-torus, a collective communication operator can be further optimized by dividing it into multiple stages, aiming to efficiently utilize the multidimensional network resources [66]. As the work [15] illustrated in Fig. 3, communication and computation can also be co-optimized to realize overlap [67], [68], while more similar co-optimized can be achieved by taking into account the barriers and degree of inter-node and intra-node parallelisms, model sharding, and pipelining designs, etc. In addition, because distributed training is sensitive to communication, processes such as I/O, P2P, and collective communication can be jointly designed with parallel strategies [18], aiming to avoid mutual interference. Such collaborative designs are effective but hard to design and realize because the designer needs to solve complex trade-offs among different parallelism, communication, and computation strategies, etc., under a given training configuration.

In terms of the above design spaces, as well as other design spaces not mentioned in this section, the cost and difficulty of exploring them in practical distributed training are significant. The studies of performance modeling and prediction for distributed training were proposed with the explicit purpose of addressing this challenge, thereby helping researchers in the exploration of intricate design spaces for distributed training.

III. PERFORMANCE MODELING AND PREDICTION FOR DISTRIBUTED DNN TRAINING

According to the methodology of modeling distributed training, the performance modeling and prediction works can be classified into the following three categories:

- *Analytical models*: utilizing analytical models or formulas to model the execution of distributed training, then predict its performance.
- *Graph-based models*: using graph-based models to describe the execution process of distributed training.
- *Execution-driven models*: combining simulation tools or real computing platforms to emulate the execution of distributed training.

This section surveys and introduces the state-of-the-art works, according to the categories above.

A. Analytical Model

The studies based on analytical models usually decompose distributed training into the computation and communication parts, then rely on analytical models to estimate their time consumption respectively. Table 1 lists recent analytical model based works, including their application coverage, the reported accuracy, and the brief descriptions of their model. The more specific models can be found in the related articles.

1) *State-of-the-Arts*: In the early work [81], Yan et al. proposed to divide the DNN model into multiple layers and segments, and then predict the training performance of DP by summing up the computation and communication time of all the segments. The main limitations of this method are the difficulties of supporting newer distributed training parallel strategy and collecting some necessary variables. To address the challenge of collecting variables, Peng et al. proposed Optimus [73], which utilizes a coefficient fitting method to model and predict the execution time of distributed training in the distributed PS architecture [82]. Specifically, Optimus formulates the analytical formula of the per-iteration time consumption, and expresses the execution time as a function of the number of PSs and GPUs, by transforming the other unknown constant terms in the formula into learnable positive coefficients.

Shi et al. proposed an analytical model [74] which can take I/O and computation-communication overlap into consideration. The work models two scenarios, i.e. when communication time is greater than computation time, and when communication time is less than or equal to computation time (less common, not shown in Table 1). As shown in Table 1, the work adds I/O time to its analytical model, and formulates computation-communication overlap by comparing the communication time and computation time during the backward propagation. In addition, on the basis of this analytical model, Kavarakunt et al. proposed an extended work [75] by taking into account factors such as mini-batch splitting time and load balancing, which is also shown in Table 1. Note that their original goals are not predicting, so their accuracy has not been reported.

In the research presented by Li et al. [76], several comprehensive analytical models are proposed for almost all the distributed training scenarios in DP. For PS with ASP, the study categorizes the modeling of distributed training into two types, i.e. the distributed training follows a First-Come-First-Serve model (*fcs*) when the network utilization of PS architecture is low, while the distributed training follows a PS model *ps* with resource sharing when the network utilization is high. For PS with BSP, the study combines *ps* and *fcs*, meanwhile taking into account the computation-communication overlap, resulting in the analytical model shown in Table 1. For the all-reduce scenario, the study directly adopts the modeling approach according to the classic model proposed by [83]. The proposed models are all experimentally validated on seven different neural networks, and finally achieve an average error of less than 1.7% on the all-reduce scenarios. The reported average error on other

TABLE I
ANALYTICAL MODEL BASED PERFORMANCE MODELING AND PREDICTION METHODS

The work.	The scope of application.	Analytical model ^a .	Reported accuracy.
[138] ^C	DP (PS+BSP).	Sort $(T_{comp}^{GPU_1}, \dots, T_{comp}^{GPU_n})$ in ascending order, let $T_{iter} = T_{comp}^{GPU_1} + T_{comm}^{push}$; for i in $(2, n)$ do: if $T_{iter} \geq T_{comp}^{GPU_i}$: then $T_{iter} = T_{comm}^{push}$; else: $T_{iter} = T_{comp}^{GPU_i} + T_{comm}^{push}$; $T_{iter} = n * T_{comm}^{pull}$; 1) n : the total number of participated GPUs. 2) T_{comm}^{push} : the total push time of a one-round iteration. 3) T_{comm}^{pull} : the total pull time of a one-round iteration.	90% to 99.4% for Alexnet [5] and VGG [111] in most cases.
AMPeD [75]	DP, tensor MP, pipeline MP, EP for transformers.	$T_{DDT} = N_{batch} * \sum_l (\frac{T_{comp}^{fwd,l} + T_{comp}^{bwd,l} + T_{comp}^{update,l}}{d_{TP} * d_{DP} * d_{PP}} + T_{comm}^{fwd,l} + T_{comm}^{bwd,l} + T_{comm}^{grad,l} + T_{bubbles}^l)$. 1) d_{TP} : the degrees of tensor MP. 2) d_{DP} : the degrees of DP. 3) d_{PP} : the degrees of pipeline MP. 4) $T_{bubbles}^l$: the waiting time of the layer l caused by the bubbles of pipelining. 5) $T_{comm}^{grad,l}$: the time costed by multi-GPU gradient synchronization of the layer l .	A maximal error of 12% on GPT [10] with a varied number of parameters.
Optimus [85]	DP (PS+BSP, PS+ASP).	$T_{iter} = \max_{PS} [b * T_{comp}^{fwd} + T_{comp}^{bwd} + T_{comm}^{pull,PS_i} + T_{comm}^{push,PS_i} + \frac{T_{comp}^{update} * N_{GPU}}{N_{PS}} + \sigma_1 * N_{GPU} + \sigma_2 * N_{PS}]$. + 1) σ_1 and σ_2 : the coefficients of communication overhead. 2) b : batch size. 3) PS_i : the i th PS.	10% average error on 5 common CNNs and 4 common RNNs.
[108] ^P	DP (PS+BSP).	$T_{iter} = T_{comm}^{I/O} + T_{comm}^{CPU_to_GPU} + T_{comp}^{fwd} + \sum_{l=C}^{N_{layer}} T_{comm}^l + \sum_{l=i}^{C-1} T_{comm}^{bwd,l} + T_{comm}^{l=1} + T_{comp}^{update,l=1}$, where $T_{comm}^l \leq T_{comm}^{bwd,l-1}$ for $l = 2, \dots, C-1$, and $T_{comm}^l > T_{comm}^{bwd,l-1}$ for $l = C, \dots, N_{layer}$.	Unreported.
[50] ^P	DP (PS+BSP).	$T_{iter} = \frac{T_{comm}^{I/O}}{\max_{i \in 1, \dots, N_{GPU}} (\sum_l T_{comp}^{fwd, GPU_i, l} + \sum_l T_{comp}^{bwd, GPU_i, l} + \sum_l T_{comm}^{mini_batch_splitting} + T_{comp}^{update})}$.	Unreported.
[62] ^P	DP (PS+BSP, PS+ASP, all-reduce).	•DP(PS+ASP): $TH_{DDT} = \sum_{i=1}^{N_{GPU}} \frac{1}{T_{comp}^{GPU_i} + resource_sharing_models}$, where $resource_sharing_models = T_{comm}^{push, GPU_i, fcs} + T_{comm}^{GPU_i, ps} + T_{comm}^{pull, GPU_i, fcs} + T_{comm}^{push, GPU_i, ps} + T_{comm}^{GPU_i, ps} + T_{comm}^{pull, GPU_i, ps}$. •DP(PS+BSP): $T_{iter} = \max(\frac{N_{GPU} * model_data_size}{2 * bandwidth}, T_{comp}^{fwd}) + \max(\frac{(N_{GPU}+1) * model_data_size}{2 * bandwidth}, T_{comp}^{bwd}) + T_{comp}^{update}$. •DP(all-reduce): $T_{iter} = T_{comp}^{fwd} + T_{comp}^{bwd} + \frac{2 * (N_{GPU}-1) * model_data_size}{N_{GPU} * bandwidth}$. 1) p : the network utilization. 2) p_{th} : a given threshold. 3) ps : the PS model. 4) fcs : the First-Come-First-Serve model. 5) TH_{DDT} : The throughput of distributed training.	•DP(PS+ASP): 19.3% error on average. •DP(PS+BSP): 13.6% error on average. •DP(all-reduce): 1.9% error on average. Evaluated on 7 common CNNs.
SMSG [123]	HP.	$Relative\ cost\ of\ training = \sum_{o_i \in O} (w * q_{x,o_i} + g * (q_{c,o_i} + q_{p,o_i})) + \sum_{e_{i,j} \in E} g * q_{r,e_{i,j}}$. 1) O : the set of all operators. 2) o_i : operator. 3) $e_{i,j}$: the dataflow from o_i to o_j . 4) E : the set of all dataflows. 5) w : the real-time computation capacities. 6) g : the real-time communication capacities. 7) q_{x,o_i} : the computation quantity of o_i . 8) q_{c,o_i} : the communication caused by parallelism of o_i . 9) q_{p,o_i} : the parameter updating communication of o_i . 10) $q_{r,e_{i,j}}$: the data redistribution quantity caused by two connected operators o_i and o_j .	Output relative results without accuracy.
FasterMoE [31] ^C	EP for transformers.	$T_{iter} = \max_{GPU_i \in GPUs} \{3 * \frac{4 * B_{GPU_i} * \alpha * H^2}{P_{GPU_i}} + 4 * \frac{B_{GPU_i} * H}{W_{net}}\}$. 1) B_{GPU_i} : the batch size of GPU_i . 2) α : fraction of intermediate embedding in MLP. 3) H : the length of the tokens' embedding vectors. 4) P_{GPU_i} : the computation throughput of GPU_i . 5) W_{net} : the communication bandwidth of inter-nodes.	R^2 scores are 0.987 to 0.967 on MoE-GPT [10] and MoE-Bert [19].
Calculon [38] ^C	HP for the transformer blocks in LLM.	Varies according to: 1) LLM configuration: the hidden size, the number of attention heads, the sequence size, the batch size, the micro-batch size, and the number of transformer blocks. 2) hardware configuration: Capacity of processors, memory system, and network. 3) execution configuration: multiple previous optimization approaches for DP, tensor MP, and pipeline MP.	Average 3.65% error and 8.87% maximum error on 22B, 175B, 530B, 1T models on Megatron [109].

^aSeveral general symbols are listed here: 1) T_z^p : the time consumption of the process z under conditions p . The process z includes whole distributed training (DDT), one training iteration ($iter$), computation ($comp$) and communication ($comm$). The p can be composed of one or more conditions, including specific GPU (GPU_i), parameter update ($update$), forward propagation (fwd), backward propagation (bwd), layer (l), push ($push$) and pull ($pull$) of PS. Some other conditions can be found in the specific method, or directly described as text. 2) N_j : the number of item j . For example, N_{GPU} stands for the total number of GPU. 3) l : the layers.

^CThe code is online available.

^PProfiling is necessary.

^EThe environment of real distributed system is needed.

Note that ^{C,P,E} are also applicable to Table 2 and Table 3.

DP scenarios is not satisfactory, so the work also proposed a fine-grained execution-driven model, which is discussed in the following section.

When a computing cluster is composed of heterogeneous GPUs with significant differences in computing power, the analytical models designed for homogeneous GPUs are no longer applicable. Therefore, the work [69] proposed by Zeng et al. introduced an analytical model that is tailored to heterogeneous computing power. In this work, each GPU's computing power is estimated by GPU frequency, the number of streaming processors, and streaming processor computing capability, in order to predict the computational time. For the communication time modeling, this work uses data volume divided by bandwidth to estimate the time of communication operations (i.e. pull and push) in PS. The work also takes into account the computation-communication overlap, so the work employs a code-based approach to predict the overall time consumption, as shown in Table I.

AMPeD [72] proposed by Moolchandani et al aims at more accurately modeling the distributed training of the transformer-based DNN [84]. For computation time, AMPeD introduces design parameters of the accelerators (such as the frequency, the number of cores, and the number of functional units) to model their throughput, then calculate the time taken by linear and non-linear computation operations. For communication time, AMPeD proposes specific communication time modeling formulas according to the communication characteristics of different modeled parallel strategies. AMPeD also provides analytical estimates for the bubble time in pipeline MP and the computation time for gradient updating. Finally, AMPeD is enabled to predict the training performance of four different parallel strategies for transformers, including DP, pipeline MP, tensor MP, and EP.

Calculon [80] presented by Mikhail et al. is an analytical performance model for training Large Language Model (LLM) in Megatron [32]. According to the given LLM configuration, hardware configuration, and execution configuration, Calculon is able to generate the performance model for training a transformer block, and then predict the batch time and memory consumption. Calculon pointed out that *other methods unnecessarily replicate the performance prediction for the transformer blocks in LLM*, so it is challenging to realize millisecond-level performance modeling for LLM training among thousands of GPUs, which Calculon can achieve. With the help of Calculon, it is possible to explore new configurations for LLM, finding ways to accelerate and optimize the distributed training process of LLM.

In order to optimize EP, He et al. proposed FasterMoE [78], which uses analytical modeling to predict the end-to-end training time consumption of each EP iteration. For the computation of EP, to compute the gradient of the inputs, a single iteration of training an MLP layer includes one General Matrix Multiplication (GEMM) in the forward propagation and two GEMM in the backward propagation. For communication, there are four times of all-to-all collective communication, including two times in each forward and backward propagation. Therefore, given a fixed network bandwidth, the training time consumption of a single iteration of EP can be modeled and predicted according

to the formula in Table I. Using this modeling as a guide, FasterMoE successfully optimized the training of the MoE model, ultimately achieving up to $17.87\times$ more efficiency for training large dynamic MoE models.

Unlike the previous work, SMSG [77] proposed by Wang et al. introduced a profile-free modeling of distributed training. SMSG proposed that it is unnecessary to estimate the actual execution time, but rather to model and compare the relative cost of different distributed training strategies. As shown in Table I, constructing relative cost only requires real computational performance and communication capacities of the participated AI accelerators, while the computational and communication quantity of distributed training can be obtained without profiling, through symbolic analysis. In this way, SMGS avoids the expensive process of actually deploying a real training environment and profiling data. Although SMGS cannot guarantee accurate modeling, it is sufficient for comparing and optimizing parallel strategies. For example, by setting the optimization objective as minimizing this relative cost, the optimal parallel strategy can be searched by comparing the costs of different parallel strategies.

In addition to the methods mentioned above, there are still several analytical models such as Cynthia [85] and the early work [81], which are not very practical for nowadays distributed training but still instructive. DTS [86] uses its built-in heuristic resources allocation algorithm (greedily maximizing the utilization of computation and communication resources in the cluster) to allocate the computing nodes and predict the execution time of pipeline MP, while [87] defines an analytical model to formulate the cost of training CNN in parallel, both of which are weak in generality.

2) *Summaries and Insights:* In summary, it can be concluded that most analytical models are targeted towards DP, with only a few analytical models being applied to Tensor MP and Pipeline MP. *The main advantages of analytical models lie in their ability to quickly and efficiently predict the computational time or throughput of distributed training.* However, analytical models have the following shortcomings:

- *Most analytical models have to introduce or profile a large number of difficult-to-obtain variables*, ultimately leading to great difficulty in deployment when the execution of distributed training is complex (e.g., the studies designed for modeling ASP [88], [89]).
- *The generality and expansibility of analytical models are not strong.* When the explored modification of design space (especially parallel strategies) affects the distributed training workflow, the analytical models must be redesigned to adapt to the newly proposed distributed training methods.

B. Graph-Based Model

Because computing graphs can well describe distributed training, it has already become a high-level Intermediate Representation (IR) for distributed training in many AI frameworks (such as TensorFlow [8] and PyTorch [10], etc.). Inspired by computing graphs, several works proposed to model the workflow of distributed training based on graphs, and then predict its performance. In Table II, we list the related works and discuss

TABLE II
GRAPH-BASED PERFORMANCE MODELING AND PREDICTION METHODS

The work.	The scope of the application.	Graph-based model.	Meaning of vertices.	Meaning of edges.	Reported accuracy.	Reported time cost.
Paleo [89] ^{C, P}	HP.	DAG.	The computation operator on a device.	The dependencies between vertices (<i>i.e.</i> a vertex cannot be executed before another is finished).	11% to 33.3% error on NiN [66], Inception [113] and Alexnet [5].	Unreported.
FlexFlow [43] ^{C, P}	HP.	Task graph.	The computation tasks or communication tasks.	The dependencies between vertices.	0% to 30% error on Inception-v3 [113] and NMT [70].	6 minutes for finding the best strategy of training NMT [70] on 16 GPUs.
Daydream [146] ^P	DP (all-reduce).	Dependency graph of kernels.	Kernel-level tasks (GPU tasks, CPU tasks, data loading tasks, communication tasks).	Dependencies of CPU tasks, GPU tasks, CUDA APIs to GPU kernels, GPU kernels to upper neural network layers, CUDA synchronization, and communications.	Less than 10% error on ResNet-50 [32], GNMT [130], and Bert [19] in most cases.	Unreported.
dPRO replayr [36] ^{P, E}	DP (PS+BSP, all-reduce).	Data flow graph.	Computation operators and fine-grained communication.	The dependencies between vertices.	less than about 6% error on Resnet-50 [32], VGG16 [111], Inception-V3 [113] and Bert-base [19].	0.04 to 0.49 hours for finding the optimal strategy on 16 GPUs.
DistSim [69] ^{P, E}	HP.	Computing graph of each node.	Computation and computation operators.	The hierarchical dependencies between vertices.	Less than 3.51% error on Bert-Large [19], GPT-2-345M [90], and T5 [92].	12.96% of time compared to direct running on a 16 GPU cluster.
Proteus [20] ^{P, E}	HP.	Distributed execution graph.	Tensor, computation operators, and communication operators.	Data dependencies and control dependencies.	3% average error on training throughput on Resnet-50 [32], Inception-V3 [113], VGG19 [111], GPT-2 [90], GPT-1.5B [90], and DLRM [76].	1.698s for VGG19 [111] and 6.265s for GPT-2 [90] in simulating one training iteration on 32 GPUs.
TAG [140]	DP (PS+BSP, all-reduce).	Computing graph.	Computation and communication operators.	The dependencies between vertices.	Unreported.	About 80 minutes on the clusters with up to 48 GPUs.

the details about how they apply graph-based models. Besides, we also summarize their reported accuracy and time cost (if mentioned in their paper), in order to help the reader further understand the performance of these works.

1) *State-of-the-Arts*: Earlier, Paleo [90] proposed by Qi et al. uses a Directed Acyclic Graph (DAG) to model distributed training. Each vertex in DAG represents a computing operator executed at the allocated computing node, including computation and I/O read/write, whose execution times are respectively estimated using the operator's computational complexity (*i.e.* Floating Point Operations, FLOPs) divided by the computation performance (*i.e.* Floating Point Operations Per Second, FLOPS) and operator memory consumption divided by the memory bandwidth of the computing node. The communication processes in distributed training are not explicitly depicted in DAG, so Paleo directly models the communication process using data amount divided by bandwidth once communication occurs between computing nodes. The edges of the DAG represent the dependency relationships between the operators, which means the directed vertex will not be executed until the previous vertex is completed. In this way, when parallelism appears, Paleo inserts synchronous boundaries into the DAG, and the time consumption of the parallel processes can be obtained by using the maximum time consumption of all DAG branches before the boundaries. Paleo finally achieves good prediction accuracy as shown in Table II, and its graph-based modeling approach has become the baseline for many subsequent studies.

Jia et al. proposed FlexFlow [93], which relies on a built-in graph-based model to automatically search for the optimal parallel strategies. Based on inputs of computing graphs, cluster

topologies, and founded parallel strategies, FlexFlow models the entire distributed training by constructing a task graph, where its vertexes stand for both computational and communication processes, and its edges represent their dependency. FlexFlow uses the same data-divide-by-bandwidth model in Paleo [90] to calculate the time consumption of communication, but the computational time is obtained by actually executing and profiling the operators. In this way, the running time of all vertexes in the task graph can be obtained, so the training throughput of distributed training under a given parallel strategy can be predicted. Furthermore, on the basis of the above model, FlexFlow is enabled to use the Metropolis-Hastings algorithm [105] to search for parallel strategies and find the optimal one.

Zhu et al. proposed DayDream [95], which uses a fine-grained graph-based model to describe distributed training, mainly targeting all-reduce based DP. DayDream uses CUPIT [106] to profile the GPU tasks, CPU tasks, data reading tasks, and communication tasks during the execution of distributed training. Then, DayDream uses the dependencies obtained by CUPIT to construct a dependency graph at the kernel level. After building such a dependency graph, DayDream is enabled to predict the execution time of distributed training. Furthermore, suppose some methods are trying to optimize the distributed training in lower levels (using methods such as Priority-Based Parameter Propagation [107], FusedAdam Optimizer [108], etc.), the optimization effect can be predicted and verified without actual deployment using DayDream, by inserting or deleting the vertexes, and modifying the edges between vertexes.

Similarly, Hu et al. proposed dPRO [98], which relies on a graph-based model to predict and optimize the performance of DP-based distributed training. DPRO consists of three modules, including a profiler, a replayer, and an optimizer. The profiler is used to collect operator-level traces on large-scale clusters, including the operators of computation and communication, and then establishes a Data Flow Graph (DFG) to describe distributed training. More specifically, the DFG consists of a local sub-DFG built by the profiled computational operators and a global sub-DFG built according to the profiled communication operators, both of which aim to draw the dependencies between the operators. Then, in its replayer, based on the generated DFG, dPRO uses the Kahn algorithm [109] to replay the trace of distributed training and predict the execution time cost. Finally, in the optimizer, dPRO identifies the bottlenecks in the DFG, and tries to relieve the bottlenecks by fusing or splitting the operators and tensors in the DFG. The optimized DFG is iteratively replayed in the replayer, aiming to search for a better DFG. According to the experiment results, dPRO can provide a maximum acceleration of 3.48 times based on the optimizer.

Another graph-based work is DistSim [99] proposed by Lu et al. which can be used to predict the detailed timeline of distributed training in each participating computing node. Based on the input DNN model and parallel strategy of HP-based distributed training, DistSim can generate a sub-computation graph for each involved computing node. Then, based on these sub-computation graphs, DistSim converts the graphs into multiple computational and communication events, and constructs the timeline of distributed training by profiled data. DistSim emphasizes that *profiling is not required for all the events (or operators), due to the large number of repetitions in the computation and communication process*. Therefore, DistSim deduplicates identical events to obtain a set of events that require profiling, and only profiles the events within the set, leading to a lower cost for predicting the entire timeline of distributed training.

To better model the runtime characteristics, Duan et al. proposed Proteus [102], which uses a policy tree to describe the parallel strategy and relies on an execution graph to model the distributed training. According to the user-provided policy tree which defined the fine-grained parallel policy for the tensors and operators, Proteus uses a compiler to generate the execution graph, by first generating tensors and computation operators, and then inserting communication operations, data dependency, and control dependency. Proteus is capable of taking into account bandwidth conflicts and computation-communication overlaps, which are two common runtime effects of distributed training. In Proteus, the time consumption of computation operations is profiled, and communication time is estimated through the $\alpha + \beta$ model [110]. Finally, Proteus can output predicted throughput and memory consumption of the distributed training, achieving approximately 3% of predicting error.

TAG [104] proposed by Zhang et al. is introduced to search for the optimal partial data parallelism [93]. Since this work proposed a strategy searching method based on the combination of Graph Neural Network (GNN) [111] and MCTS [112], TAG also relies on a graph-based model to describe the distributed

training process. More specifically, for the aim of reducing communication, TAG design a GNN-based reinforcement learning approach to guide MCTS, finding the possible cut in the computation graph then replacing the original all-reduce or PS communication as sufficient factor broadcasting [113]. After such an optimization, TAG transforms the optimized computation graph into a distributed training graph, and inserts necessary auxiliary operators to ensure the authenticity of the graph-based model. Then, according to the dependency described by the distributed training graph, TAG sets up a FIFO-queues base virtual runtime to evaluate the optimized computation graph using the profiled data. In this way, TAG can optimize the communication overhead and speed up the distributed training, both for unseen cluster topology and DNN models without retraining.

2) *Summaries and Insights*: Compared with the analytical models, *graph-based models can model more details about the workflow of distributed training*, where the reasons are as follows:

- *Graphs offer the ability to model dependencies*, so any modification on the graph-based model can be more correctly mapped to the workflow changes of distributed training, leading to better accuracy.
- *On graph-based models, distributed training can be modeled and optimized through making graph operations*. For instance, dPRO [98] uses vertex fusing and splitting to achieve training acceleration, while TAG [104] relies on graph cuts to find possible optimization.

However, graph-based models also have their limitations:

- As shown in Table II, most works use profiled data for computation and communication modeling, which *requires actual devices or even an expensive real distributed training environment*.
- *The size and complexity of the generated graph increase rapidly as the scale of the modeled computing cluster increases*, posing significant challenges to predicting speed and memory consumption.

C. Execution-Driven Model

In both analytical and graph-based models, it is usually necessary to predefine the execution flow of distributed training, leading to significant challenges in modeling the runtime characteristics. In contrast, execution-driven models are realized by building self-designed simulators or combining highly realistic domain-specific simulators and real running environments, thereby providing a more realistic modeling capability for the runtime of distributed training. We summarize the state-of-the-art execution-driven models in Table III.

1) *State-of-the-Arts*: Without actual processing, it is hard to obtain the runtime execution details of ASP-based PS. To solve this problem, in the work [114], Lin et al. proposed to use the queueing model and Mean Value Analysis (MVA) [115] to model its runtime execution and predict its throughput. This work utilized a simple architecture consisting of one PS and one worker to profile the uplink and downlink transmission time in the PS architecture, as well as the time consumption for gradient computation and updates. These profiling results

TABLE III
EXECUTION-DRIVEN PERFORMANCE MODELING AND PREDICTION METHODS

The work.	The scope of the application.	Reported accuracy.	Reported time cost.
[67] ^P	DP (PS+ASP).	An average error of 4% to 14% on CNNs and Inception-V3 [113].	Unreported.
[61] ^P	DP (PS+ASP).	Within 20% predicting error on most case of Inception [113], Resnet [32], and VGG [111].	About 5% to 10% of the real execution time for 10 workers.
[62] ^P	DP (PS+BSP, PS+ASP, all-reduce).	•DP (PS+ASP): 1.8% error on average. •DP (PS+BSP): 3% error on average. •DP (all-reduce): 1.7% error on average. Evaluated on 7 common CNNs.	Unreported.
PerfEstimator [133] ^P	DP (all-reduce).	Less than 3% error on VGG13 [111] and less than 11% error on Resnet-50 [32].	Unreported.
DNNEmu [124] ^P	DP (BSP).	Less than 10.9% error on 5 common CNNs.	Unreported.
DistIR [104]	HP.	0.9289 for the Person coefficient and 0.9311 for the Spearman coefficient on MLP models.	Processing over 16000 ops under a second.
ASTRA-Sim [96, 128] ^C	HP.	Output relative results without accuracy.	Running 1MB all-reduce for 64 GPUs takes 21 minutes.

were then used as inputs for its queueing modeling, in order to emulate the runtime execution of ASP-based PS. Using a similar profiling method, the work [116] proposed by Li et al. design a discrete event simulator to synthesize the workloads of ASP-based PS. This work emphasized that profiling errors may caused by the gRPC over HTTP/2 used by TensorFlow, so it proposed to correct the data of the profiled communication operator before using it to generate the workloads and predict the training iteration time. Furthermore, building upon the work in [116], Li et al. also extended the simulation capabilities of the proposed simulator in [76], by incorporating simulation support, and finally achieved a much better accuracy as shown in Table III.

Yang et al. proposed perfEstimator [117], which combines simulation and actual execution to model and predict the time consumption of distributed training with all-reduce based DP. More specifically, perfEstimator trains the DNN model on a single computing node, and uses a profiling tool to obtain the actual time cost for each computation operator. For communication, perfEstimator utilizes Simgrid [118] to simulate and obtain the time consumption of collective communication. In order to model the computational performance (i.e. batch/s), PerfEstimator defines a scaling factor α to formulate the performance loss introduced by adding n computing nodes, where

$$\alpha = \frac{OP_{bp,end_time}^{l1} - OP_{fp,start_time}^{l1}}{OP_{all-reduce,end_time}^{l1} - OP_{fp,start_time}^{l1}}, \quad (1)$$

$OP_{a,t}^{l1}$ stand for the moment t of the first layer operator finishes its execution process a . Then, given p_1 as the training

performance of a single node without distribution, the computational performance p_n with n nodes can be represented as $p_n = p_1 * n * \alpha$, which can be used to predict the iteration time of distributed training.

Similarly, DNNEmu [119] proposed by Wang et al. uses CPU to emulate the computation process of distributed training, enabling users to explore distributed communication strategies without considering the computational details and hardware environment. By profiling, DNNEmu first collects and builds an operator data set about their computation time and the related features. Given a target operator, if the operator is included in the profiled data set, the recorded computation time can be directly used. Otherwise, a Gradient Boosting Decision Tree (GBDT) [120] trained based on the profiled data would be applied to predict the computation time for the operator. In this way, DNNEmu can obtain the computation time for any operator in any neural network. Subsequently, DNNEmu modifies MXNet [121], using sleep functions to emulate the execution of those operators, without changing the other processes of running distributed training. Similar studies using learning-based models to predict the computation time of training DNN models also include [122], [123], [124], although they are designed for single-node training rather than distributed training.

Santhanam et al. introduced DistIR [125], an intermediate representation based on MLIR [126] to model the execution process of distributed training. In DistIR, the distributed training process is represented in pseudocode, where each operator is defined as a function with specified inputs, outputs, and computing nodes for execution. Then, in the DistIR-based simulator, each execution computing node is modeled as a single-threaded execution unit, allowing operators to be executed one by one in accordance with the code order. The operators not belonging to the same execution computing node can be executed in parallel once they satisfy their input dependency conditions, and their time and memory consumption can be predicted using either analytical models or profiled data. Leveraging the simulation capability of the DistIR simulator, researchers can explore the performance of parallel strategies within a complex search space.

To provide a simulator that helps researchers understand and explore the software/hardware co-design spaces of distributed training, Rashidi et al. proposed ASTRA-Sim [66], which is realized by integrating multiple mature simulators. ASTRA-Sim consists of three layers, including workload, system, and network layer. The workload layer describes the computation time, the details of collective communication, and the data transfer volume for each neural layer in the forward and backward propagation. ASTRA-Sim uses Scale-sim [127] to obtain the computation time, therefore ASTRA-Sim is able to simulate distributed training without relying on any real computing node. The system layer describes the scheduling strategy (e.g. FIFO or LIFO) for the collective communication during distributed training. The network layer allows for defining the network topology of the cluster, and feeds the collective communication from the workload layer into network simulators, such as Garnet [128] or Ns-3 [129], for simulating the communication process. Based on the cooperation of the above three layers, ASTRA-Sim is capable of not only getting the computation time of each

TABLE IV
EXPERIMENT RESULT OF DIFFERENT MODELING METHODS UNDER THE SAME BENCHMARK

Experiment settings								
Hardware: Two servers equipped with NVIDIA Quadro RTX4000, connected by 1GbE network connection.								
Software: Ubuntu 18.04, python 3.7, CUDA 10.2.								
Training dataset: ImageNet dataset with 3x224x224 data input.								
Actual execution			Modeling and predicting					
Neural network	Batch size	Average iteration time of execution	Modeling and predicting method	Model category	Predicted iteration time	Accuracy	Predicting time cost	Normalized predicting time cost*
Alexnet	32	8.38s	The work [138]	Analytical	7.67s	91.5%	2.13us	$\times 2.78 \times 10^{-7}$
			Paleo [89]	Graph-based	7.61s	90.8%	4.95ms	$\times 6.50 \times 10^{-4}$
			Astra-sim [96]	Execution-driven	9.03s	92.7%	136.6s	$\times 15.1$
	64	8.98s	The work [138]	Analytical	7.45s	82.9%	2.14us	$\times 2.87 \times 10^{-7}$
			Paleo [89]	Graph-based	8.01s	89.2%	5.17ms	$\times 6.45 \times 10^{-4}$
			Astra-sim [128]	Execution-driven	9.95s	90.3%	138.4s	$\times 13.9$

* The normalized predicting time cost is the predicting time cost divided by the average iteration time of execution.

neural layer, but also providing detailed time consumption and bottleneck analysis of the communication process under a given design space. Furthermore, in its subsequent work ASTRA-Sim 2.0 [65] proposed by Won et al. a trace-based simulation [130], an analytical network simulation, a memory model, and more network topology are supported, resulting in more supported design space and better simulation capabilities.

2) *Summaries and Insights:* In conclusion, the execution-driven model has the following advantages:

- *The execution-driven model is able to simulate complex runtime characteristics*, which enables execution-driven models to model more details of distributed training.
- *The recent execution-driven model exhibits a highly modular characteristic*, thus allowing the exploration of difference design space through individually modifying the concerned modules (e.g. the computation and memory modules in DistIR [125], the schedule strategy for the collective communication in Astra-sim [66]).

Nevertheless, *most execution-driven model inevitably meets the expense of simulation time consumption and computational overhead when the modeling scale is large*, making execution-driven models the least efficient and scalable compared with the other two categories.

IV. MODEL ANALYSIS AND SELECTION

To assist in selecting and applying the models above, this section further conducts experimental comparisons, analyzes the application scenarios, and proposes a step-by-step guide for choosing the appropriate modeling and prediction method.

A. Performance Comparison and Analysis

Due to the absence of performance testing under the same benchmark for previous work, as supplementary, we select and implement the work [69], paleo [90], Astra-sim [66], and then conduct a fair experiment to compare their performance. The reasons for choosing them are:

- They are representative works within the three categories of modeling and prediction methods.

- They are general works where their implementations are not limited to the architecture of the neural networks to be trained.
- They have well-maintained end-to-end public code and do not rely on complex experimental environments for profiling, which helps quickly reproduce, deploy, and apply for users.

The experiment settings and results are shown in Table IV, where the results include the measured iteration time of actually executing distributed training, the accuracy of modeling and predicting, and the predicting time cost of each method.

The experiment results are basically in line with the expectations of Section III. In terms of accuracy, the overall trend indicates that the performance of the execution-driven model is the best, followed by the graph-based model, with the analytical model showing a lower performance. Because of the integration of some analytical models for computation and communication processes, the accuracy of Paleo is close to the work [69] when the batch size is 32. Regarding the predicting time cost, it can be found that the execution-driven model requires more than tens of thousands of times the predicting time cost compared to the graph-based model, and more than tens of millions of times the predicting time cost compared to the analytical model. Note that the predicting time cost of the execution-driven model is comparable and even more than the actual execution of distributed training, and if fine-grained simulations such as Scale-sim [127] or Garnet [128] are introduced in the simulator backend of Astra-Sim, or the distributed training scale becomes larger, the predicting time cost will further increase significantly.

B. Application Scenario for Different Categories of Models

Based on the above experiment results, we further analyzed the application scenarios for different categories of models.

1) *Analytical Models:* Although analytical models can hardly match the accuracy of the other two categories, their predicting time cost is extremely small. Therefore, besides the application scenario of performance analysis and prediction,

analytical models can also be applied to dynamically optimize distributed training. Specifically, most analytical models use mathematical formulas to predict the time consumption of distributed training, so their predicting time cost would not increase with the scale of the distributed training. Therefore, as shown in Table IV, the predicting time cost of analytical models would be much shorter than the per-iteration execution time of distributed training, so it can be integrated into the actual distributed training process to achieve dynamical training speed optimization, by combining searching or optimizing methods such as [131].

2) *Graph-Based Models:* Given their ability to clearly model dependencies, *graph models are more suitable for modeling and optimizing parallel strategies.* On the one hand, when designing parallel strategies, by modifying the elements of a graph such as vertices and edges, graph models can effectively describe the changes in computational and communication processes. On the other hand, graphs offer good interpretability, helping researchers understand the actual execution flow of the modeled parallel strategies. With the help of the above advantages, works such as FlexFlow [93] and Daydream [95] have successfully applied graph models to optimize parallel strategies, and have achieved notable results.

3) *Execution-Driven Models:* As the large predicted time cost shown in Table IV, execution-driven models can hardly be applied in dynamic or online scenarios. However, *execution-driven models are more suitable for static application scenarios such as system design that is insensitive to latency.* The greatest advantage of execution models lies in their ability to model fine-grained design spaces, and their high sensitivity to various design parameters or variations within these design spaces. By leveraging the characteristics of execution models, researchers are enabled to explore or optimize multiple design spaces such as cluster design, memory and data loading, computation, communication, and even their collaborative design. Several existing studies [132], [133] have successfully utilized execution models to optimize their system design for distributed training, which well validates the effectiveness of execution models.

C. Steps to Select Suitable Modeling and Predicting Method

We provide a guide for choosing an appropriate modeling or prediction method as follows:

- Begin by identifying the right category of models according to the application scenario discussed in Section IV-B.
- Filter and identify the methods according to the expected parallel strategy. If the parallel strategy is uncertain, it is better to choose the methods that support HP.
- Prioritize methods that match the predicting time cost requirements and have higher reported accuracy.
- For those who aiming for a quick start, methods that have available code and do not necessitate profiling or a real distributed system environment should be better choices (i.e. labeled as C independently in Tables I–III).
- If no existing work perfectly fits the requirements, consider modifying a similar work that has available code, which would save development costs and time.

V. CHALLENGES AND FUTURE OPPORTUNITIES

Despite the numerous efforts dedicated to modeling and predicting the performance of distributed DNN training, there are still several limitations in existing research, resulting in areas that can be further improved. This section aims to discuss the current challenges and future opportunities, taking into account the findings from the existing state-of-the-art works.

A. Fine-Grained Modeling of Computing and Memory

1) *Challenges:* As research on distributed training using heterogeneous computing [134], [135] continues to increase and become commercially viable, there is an urgent need for fine-grained and high-precision modeling for computing nodes. Furthermore, innovations in memory system architecture, such as memory pooling [136] and memory disaggregation [55], also present new challenges for modeling the computing process. However, existing modeling or simulation methods that use the constant value of FLOPS without considering memory often suffer from deviations, while profiling-based methods inevitably require an actual hardware environment. As a result, the existing methods can hardly achieve fine-grained modeling for the computing and memory of distributed training.

2) *Opportunities:* One solution is to integrate the model of distributed training with existing GPU architecture simulators such as GPGPU-Sim [137] and Gem5-APU [138], or combine memory simulators such as DRAMsim3 [139], both of which help construct high-precision modeling of GPU computing architecture. Astra-sim [66] provides a good example by combining Scale-sim [127] or execution traces [130] to model distributed training, allowing Astra-sim to work more similarly with the real hardware environment in terms of computing and memory. Nevertheless, it is important to note that those fine-grained simulators usually operate at the cycle level, so the simulation speed is also a key factor that needs to be further considered.

B. Support for Complex Network Topologies

1) *Challenges:* The network topology between computing nodes has a significant impact on distributed training. However, existing studies have limited capabilities in modeling network topologies. Most existing works [90], [102] rely on simple analytical models (e.g., data volume/bandwidth, $\alpha + \beta$ models) or profiling data (e.g. Daydream [95] and dPRO [98]), which can hardly model the network behaviors such as congestion control. As a result, if researchers want to evaluate the impact of network topologies on distributed training and explore the optimal interconnection topology of computing nodes before deployment, existing works may not meet their requirements.

2) *Opportunities:* Fortunately, many existing network simulation tools already support customizing network topologies. For example, Simgrid [118] allows the description of custom topologies with bandwidth, delay, and routing information in XML files. Ns-3 [129] and OPNET [140] also allow fine-grained modifications of network topologies in simulations. Therefore, it is possible to integrate the network communication processes

of distributed training into mature network simulation tools, enhancing the modeling capabilities for network topologies. It is important to note that the choice of network simulator also determines the scalability of modeling distributed training. A coarser-grained network simulator allows for better scalability, while a fine-grained network simulator is more suitable for modeling small-scale distributed training.

C. Unified Standard Benchmark for Performance Prediction

1) *Challenges*: Although existing works claimed their high accuracy and had reported their experimental verification, both the methods of evaluation and the DNNs used for comparison are different, making it difficult to make cross-comparisons for these works. In this case, there is an urgent need for a standardized and unified benchmark to evaluate the performance of different modeling and predicting approaches.

2) *Opportunities*: For the establishment of such a benchmark, reference can be taken from the benchmarking works in the field of AI, such as MLPerf [141], to create a unified set of basic tests that include various distributed training configurations and training tasks with commonly used DNNs. Then, the constructed benchmark can be used to evaluate the accuracy and execution performance of different performance modeling and predicting works. Building a unified benchmark is helpful for the users to choose more accurate and efficient performance-predicting tools, and it can also assist researchers in continuously iterating and optimizing their works in this field. Chakra [130] provides a good method to generate execution traces of distributed training for benchmarking.

D. Adaptability to More Design Spaces

1) *Challenges*: As AI technology continues to evolve, innovation and optimization for design spaces will keep emerging. In this case, the performance modeling and prediction studies also need to keep up with the changes, by designing and realizing adaptability to new design spaces, which means they should support correctly working after opening, closing, or modifying a part of their design spaces in the model.

2) *Opportunities*: In terms of adaptability to new tasks, graph-based and execution-driven models significantly show their advantage. For example, by changing the interconnections between nodes and edges in existing graph-based models (e.g. Paleo [90]), the design space of a new parallel strategy can be expressed, without making modifications to other design spaces such as computation and communication. Leveraging modular characteristics, the recent execution-driven models also support inserting, removing, or switching components of their models [66], [125]. In contrast, the analytical models can hardly adapt to new distributed training, due to their fixed performance modeling formula. To enhance the adaptability of existing works, the performance modeling and predicting works need to provide users with flexible and configurable interfaces to configure the design spaces, without causing modeling failure. It is also promising to combine well-verified analytical models with the other two, especially when modeling design spaces such as computation or communication.

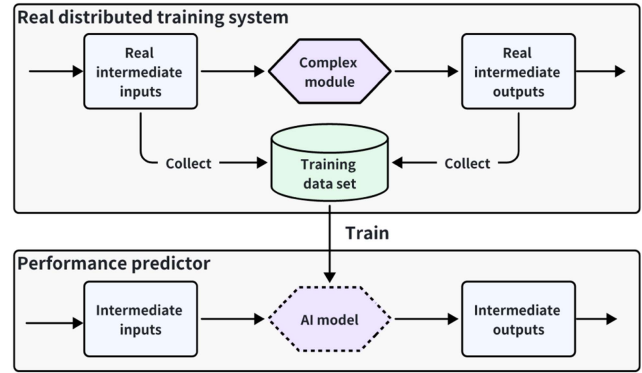


Fig. 4. Illustration for performance modeling and predicting for distributed training using AI models.

E. AI-Enabled Modeling, Predicting, and Dynamic Optimizing

1) *Challenges*: Due to the vast design spaces and numerous factors of distributed training, taking into account all the details through fine-grained modeling would improve prediction accuracy but severely degrade the prediction speed. In such cases, modeling large-scale distributed training would become notably slow, and optimizing distributed training during runtime is also impractical, both of which limit the application of the modeling and prediction studies.

2) *Opportunities*: To address this problem, insights can be drawn from several simulation works that incorporated AI models as the foundation for modeling or simulation [142], [143], [144], especially in the field of simulating large-scale computing or communication systems. Following this idea, as shown in Fig. 4, by training an AI model based on the collected datasets from a real-world system, a well-trained AI model can serve as a black-box model, replacing several compute-intensive modules to achieve efficient and accurate performance predicting. For example, in the work DNNemu [119], GBDT is used to predict the computational process, which provides good insights for AI-based modeling. Researchers can choose to replace the intermediate modules they are less concerned about using AI models, while utilizing high-fidelity and interpretable models for the modules they are more interested in.

Furthermore, AI is promising to achieve dynamic optimization for distributed training. Because distributed training produces highly repetitive patterns at fine granularity, it becomes possible to predict its real-time performance by profiling a few training iterations. Then, complementary to prediction, AI-driven dynamic optimization approaches can be applied to boost the performance of distributed training. For instance, DL² [145] employs reinforcement learning to dynamically optimize the configuration of each training task, Spotlight [146] solves the device placement problem of distributed training by combining Markov decision process and reinforcement learning. Although these existing works can hardly optimize complex distributed training strategies such as HP, they have proven that AI-driven approaches are effective and fast enough to achieve real-time optimization for distributed training, rather than pre-computed before the actual execution.

VI. CONCLUSION

Because deploying distributed training is time-consuming and expensive, modeling and predicting the performance has become one of the most effective ways to help optimize the design space of distributed training. This paper first introduces the background and significance of performance modeling and prediction for distributed DNN training, and then discusses the main design spaces of distributed training. Subsequently, the cutting-edge related works are classified into three categories, including analytical models, graph-based models, and execution-driven models, and then the key ideas, strengths, and weaknesses of the different categories are further introduced. This paper also makes fair experiments to compare the performance of the different categories of methods, and analyze their application scenarios and selection methods. Finally, this article discusses the challenges of existing works, and also proposes some possible opportunities and solutions for improving the works in this research field. It is hoped that this article can help understand the principles, capabilities, and application range of the studies on performance modeling and prediction for distributed DNN training. Additionally, this article also aims to motivate researchers to propose more efficient and practical research outcomes for modeling and predicting the performance of distributed DNN training, thereby promoting the development and evolution of AI technology.

REFERENCES

- [1] S. Zhang et al., "OPT: Open pre-trained transformer language models," 2022, *arXiv:2205.01068*.
- [2] A. Zeng et al., "GLM-130b: An open bilingual pre-trained model," 2022, *arXiv:2210.02414*.
- [3] OpenAI, 2022. [Online]. Available: <https://openai.com/blog/chatgpt/>
- [4] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
- [5] A. Kirillov et al., "Segment anything," 2023, *arXiv:2304.02643*.
- [6] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.
- [7] W. X. Zhao et al., "A survey of large language models," 2023, *arXiv:2303.18223*.
- [8] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous distributed systems," 2016, *arXiv:1603.04467*.
- [9] N. Shazeer et al., "Mesh-tensorflow: Deep learning for supercomputers," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 10435–10444.
- [10] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 8026–8037.
- [11] Amazon, 2023. [Online]. Available: <https://calculator.aws/>
- [12] J. W. Rae et al., "Scaling language models: Methods, analysis & insights from training gopher," 2021, *arXiv:2112.11446*.
- [13] Website, 2023. [Online]. Available: <https://www.statista.com/statistics/1384401/energy-use-when-training-llm-models/>
- [14] PyTorch, "Training a 1 trillion parameter model with PyTorch fully sharded data parallel on AWS," website, 2022. [Online]. Available: <https://medium.com/pytorch/training-a-1-trillion-parameter-model-with-pytorch-fully-sharded-data-parallel-on-aws-3ac13aa96c6f>
- [15] Y. Peng et al., "A generic communication scheduler for distributed DNN training acceleration," in *Proc. ACM Symp. Operating Syst. Princ.*, 2019, pp. 16–29.
- [16] J. Verbracke et al., "A survey on distributed machine learning," *ACM Comput. Surv.*, vol. 30, pp. 1–33, 2020.
- [17] R. Mayer et al., "Scalable deep learning on distributed infrastructures: Challenges, techniques, and tools," *ACM Comput. Surv.*, vol. 53, pp. 1–37, 2020.
- [18] Z. Tang et al., "Communication-efficient distributed deep learning: A comprehensive survey," 2020, *arXiv: 2003.06307*.
- [19] M. Li et al., "Communication efficient distributed machine learning with the parameter server," in *Proc. Adv. Neural Inf. Process. Syst.*, 2014, pp. 19–27.
- [20] A. Sergeev et al., "Horovod: Fast and easy distributed deep learning in tensorflow," 2018, *arXiv: 1802.05799*.
- [21] P. Patarasuk and X. Yuan, "Bandwidth efficient all-reduce operation on tree topologies," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2007, pp. 1–8.
- [22] A. V. Gerbessiotis et al., "Direct bulk-synchronous parallel algorithms," *J. Parallel Distrib. Comput.*, vol. 22, pp. 251–267, 1994.
- [23] Q. Ho et al., "More effective distributed ML via a stale synchronous parallel parameter server," in *Proc. Adv. Neural Inf. Process. Syst.*, 2013, pp. 1223–1231.
- [24] B. Recht et al., "HOGWILD!: A lock-free approach to parallelizing stochastic gradient descent," in *Proc. Adv. Neural Inf. Process. Syst.*, 2011, pp. 693–701.
- [25] Y. Jiang et al., "A unified architecture for accelerating distributed DNN training in heterogeneous GPU/CPU clusters," in *Proc. 14th USENIX Conf. Operating Syst. Des. Implementation*, 2020, pp. 463–479.
- [26] Y. Huang et al., "GPipe: Efficient training of giant neural networks using pipeline parallelism," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 103–112.
- [27] S. Fan et al., "DAPPLE: A pipelined data parallel approach for training large models," in *Proc. ACM SIGPLAN Symp. Princ. Pract. Parallel Program.*, 2021, pp. 431–445.
- [28] S. Li et al., "Chimera: Efficiently training large-scale neural networks with bidirectional pipelines," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2021, pp. 1–14.
- [29] S. Rajbhandari et al., "DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation ai scale," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 18332–18346.
- [30] Z. Chen et al., "Towards understanding mixture of experts in deep learning," 2022, *arXiv:2208.02813*.
- [31] J. Rasley et al., "DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 3505–3506.
- [32] M. Sholehybi et al., "Megatron-LM: Training multi-billion parameter language models using model parallelism," 2019, *arXiv: 1909.08053*.
- [33] P. Liang et al., "A survey on auto-parallelism of large-scale deep learning training," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 8, pp. 2377–2390, Aug. 2023.
- [34] M. Wang et al., "Supporting very large models using automatic dataflow graph partitioning," in *Proc. 14th EuroSys Conf.*, 2019, pp. 1–17.
- [35] L. Zheng et al., "Alpa: Automating inter-and intra-operator parallelism for distributed deep learning," in *Proc. 16th USENIX Symp. Operating Syst. Des. Implementation*, 2022, pp. 559–578.
- [36] S. Li et al., "Colossal-AI: A unified deep learning system for large-scale parallel training," in *Proc. Int. Conf. Parallel Process.*, 2023, pp. 766–775.
- [37] Z. Cai et al., "TensorOpt: Exploring the tradeoffs in distributed DNN training with auto-parallelism," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 8, pp. 1967–1981, Aug. 2022.
- [38] C. Unger et al., "Unity: Accelerating DNN training through joint optimization of algebraic transformations and parallelization," in *Proc. 16th USENIX Symp. Operating Syst. Des. Implementation*, 2022, pp. 267–284.
- [39] P. Micikevicius et al., "Mixed precision training," 2017, *arXiv: 1710.03740*.
- [40] Q. Ye, Y. Zhou, M. Shi, Y. Sun, and J. Lv, "DLB: A dynamic load balance strategy for distributed training of deep neural networks," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 7, no. 4, pp. 1217–1227, Aug. 2023.
- [41] Z. Jia et al., "TASO: Optimizing deep learning computation with automatic generation of graph substitutions," in *Proc. ACM Symp. Operating Syst. Princ.*, 2019, pp. 47–62.
- [42] T. Chen et al., "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proc. 13th USENIX Conf. Operating Syst. Des. Implementation*, 2018, pp. 578–594.
- [43] Y. Ueno and R. Yokota, "Exhaustive study of hierarchical allreduce patterns for large messages between GPUs," in *Proc. 19th IEEE/ACM Int. Symp. Cluster, Cloud Grid Comput.*, 2019, pp. 430–439.
- [44] W. Won et al., "TACOS: Topology-aware collective algorithm synthesizer for distributed training," 2023, *arXiv:2304.05301*.
- [45] S. Agarwal et al., "On the utility of gradient compression in distributed training systems," in *Proc. Mach. Learn. Syst.*, vol. 4, pp. 652–672, 2022.
- [46] J. Romero et al., "Accelerating collective communication in data parallel training across deep learning frameworks," in *Proc. 19th USENIX Symp. Netw. Syst. Des. Implementation*, 2022, pp. 1027–1040.

- [47] S. Rajbhandari et al., "ZeRO: Memory optimizations toward training trillion parameter models," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2020, pp. 1–16.
- [48] J. Ren et al., "ZeRO-offload: Democratizing billion-scale model training," in *Proc. USENIX Annu. Tech. Conf.*, 2021, pp. 551–564.
- [49] S. Rajbhandari et al., "ZeRO-infinity: Breaking the GPU memory wall for extreme scale deep learning," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2021, pp. 1–14.
- [50] P. Jain et al., "Checkmate: Breaking the memory wall with optimal tensor rematerialization," in *Proc. Mach. Learn. Syst.*, vol. 2, pp. 497–511, 2020.
- [51] T. Chen et al., "Training deep nets with sublinear memory cost," 2016, *arXiv:1604.06174*.
- [52] V. A. Korthikanti et al., "Reducing activation recomputation in large transformer models," in *Proc. Mach. Learn. Syst.*, 2023, pp. 341–353.
- [53] A. Jain, A. Phanishayee, J. Mars, L. Tang, and G. Pekhimenko, "Gist: Efficient data encoding for deep neural network training," in *Proc. ACM/IEEE 45th Annu. Int. Symp. Comput. Architecture*, 2018, pp. 776–789.
- [54] J. Evans, "Nvidia grace," in *Proc. IEEE Hot Chips 34 Symp.*, 2022, pp. 1–20.
- [55] D. Gouk et al., "Direct access, high-performance memory disaggregation with DirectCXL," in *Proc. USENIX Annu. Tech. Conf.*, 2022, pp. 287–294.
- [56] X. Gu and C. Ding, "A generalized theory of collaborative caching," in *Proc. Int. Symp. Memory Manage.*, 2012, pp. 109–120.
- [57] J. Liu et al., "Lobster: Load balance-aware I/O for distributed DNN training," in *Proc. Int. Conf. Parallel Process.*, 2022, pp. 1–11.
- [58] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The hadoop distributed file system," in *Proc. IEEE 26th Symp. Mass Storage Syst. Technol.*, 2010, pp. 1–10.
- [59] J. Mohan et al., "Analyzing and mitigating data stalls in DNN training," 2020, *arXiv:2007.06775*.
- [60] M. Kuchnik et al., "Plumber: Diagnosing and removing performance bottlenecks in machine learning data pipelines," in *Proc. Mach. Learn. Syst.*, 2022, pp. 33–51.
- [61] N. A. Gawande et al., "Scaling deep learning workloads: Nvidia dgx-1/pascal and intel knights landing," *Elsevier Future Gen. Comput. Syst.*, vol. 108, pp. 1162–1172, 2020.
- [62] K. Arnold et al., "Application-specific hardware accelerators," 2019. [Online]. Available: <https://engineering.fb.com/datacenter-engineering/accelerating-infrastructure>
- [63] N. P. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," in *Proc. Annu. Int. Symp. Comput. Archit.*, 2017, pp. 1–12.
- [64] T.-N. Truong et al., "Hybrid electrical/optical switch architectures for training distributed deep learning in large-scale," *IEICE Trans. Inf. Syst.*, vol. E104-D, pp. 1332–1339, 2021.
- [65] W. Won, T. Heo, S. Rashidi, S. Sridharan, S. Srinivasan, and T. Krishna, "ASTRA-sim2.0: Modeling hierarchical networks and disaggregated systems for large-model training at scale," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw.*, 2023, pp. 283–294.
- [66] S. Rashidi, S. Sridharan, S. Srinivasan, and T. Krishna, "Astra-sim: Enabling SW/HW co-design exploration for distributed DL training platforms," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw.*, 2020, pp. 81–92.
- [67] S. Rashidi et al., "Enabling compute-communication overlap in distributed deep learning training platforms," in *Proc. ACM/IEEE 48th Annu. Int. Symp. Comput. Architecture*, 2021, pp. 540–553.
- [68] S. Wang, A. Pi, X. Zhou, J. Wang, and C.-Z. Xu, "Overlapping communication with computation in parameter server for scalable DL training," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 9, pp. 2144–2159, Sep. 2021.
- [69] Y. Zeng et al., "Modeling the training iteration time for heterogeneous distributed deep learning systems," *Int. J. Intell. Syst.*, vol. 2023, no. 1, 2023, Art. no. 2663115.
- [70] M. Z. Alom et al., "The history began from alexnet: A comprehensive survey on deep learning approaches," 2018, *arXiv:1803.01164*.
- [71] K. Simonyan et al., "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.
- [72] D. Moolchandani, J. Kundu, F. Ruelens, P. Vranckx, T. Evenblij, and M. Perumkunnil, "AMPed: An analytical model for performance in distributed training of transformers," in *IEEE Int. Symp. Perform. Anal. Syst. Softw.*, 2023, pp. 306–315.
- [73] Y. Peng et al., "Optimus: An efficient dynamic resource scheduler for deep learning clusters," in *Proc. EuroSys Conf.*, 2018, pp. 1–14.
- [74] S. Shi, Q. Wang, and X. Chu, "Performance modeling and evaluation of distributed deep learning frameworks on GPUs," in *Proc. IEEE 16th Int. Conf. Dependable, Autonom. Secure Comput., 16th Int. Conf. Pervasive Intell. Comput., 4th Int. Conf. Big Data Intell. Comput. Cyber Sci. Technol. Cong.*, 2018, pp. 949–957.
- [75] T. Kavarakuntla, L. Han, H. Lloyd, A. Latham, and S. B. Akintoye, "Performance analysis of distributed deep learning frameworks in a multi-GPU environment," in *Proc. IEEE 20th Int. Conf. Ubiquitous Comput. Commun.*, 2021, pp. 406–413.
- [76] Z. Li, M. Paolieri, L. Golubchik, S.-H. Lin, and W. Yan, "Predicting throughput of distributed stochastic gradient descent," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 2900–2912, Nov. 2022.
- [77] H. Wang et al., "MSG: Profiling-free parallelism modeling for distributed training of DNN," *Springer Int. J. Parallel Program.*, vol. 51, pp. 109–127, 2023.
- [78] J. He et al., "Fastermoe: Modeling and optimizing training of large-scale dynamic pre-trained models," in *Proc. ACM SIGPLAN Symp. Princ. Pract. Parallel Program.*, 2022, pp. 120–134.
- [79] J. Devlin et al., "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.
- [80] M. Isaeiev et al., "Calculon: A methodology and tool for high-level co-design of systems and large language models," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2023, pp. 1–14.
- [81] F. Yan et al., "Performance modeling and scalability optimization of distributed deep learning systems," in *ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2015, pp. 1355–1364.
- [82] W. Zhao et al., "Distributed hierarchical GPU parameter server for massive scale deep learning ADS systems," in *Proc. Mach. Learn. Syst.*, 2020, pp. 412–428.
- [83] P. Patarasuk et al., "Bandwidth optimal all-reduce algorithms for clusters of workstations," *J. Parallel Distrib. Comput.*, 2009, pp. 117–124.
- [84] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
- [85] H. Zheng et al., "Cynthia: Cost-efficient cloud resource provisioning for predictable distributed deep neural network training," in *Proc. Int. Conf. Parallel Process.*, 2019, pp. 1–11.
- [86] W. J. Robinson M., F. Esposito, and M. A. Zuluaga, "DTS: A simulator to estimate the training time of distributed deep neural networks," in *Proc. 30th Int. Symp. Model., Anal., Simul. Comput. Telecommun. Syst.*, 2022, pp. 17–24.
- [87] Z. Jia et al., "Exploring hidden dimensions in accelerating convolutional neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 2274–2283.
- [88] Z. Pei, C. Li, X. Qin, X. Chen, and G. Wei, "Iteration time prediction for CNN in multi-GPU platform: Modeling and analysis," *IEEE Access*, vol. 7, pp. 64788–64797, 2019.
- [89] Y. Oyama, A. Nomura, I. Sato, H. Nishimura, Y. Tamatsu, and S. Matsuoka, "Predicting statistics of asynchronous SGD parameters for a large-scale distributed deep learning system on GPU supercomputers," in *Proc. IEEE Int. Conf. Big Data*, 2016, pp. 66–75.
- [90] H. Qi, E. R. Sparks, and A. Talwalkar, "Paleo: A performance model for deep neural networks," in *Proc. Int. Conf. Learn. Representations*, 2022.
- [91] M. Lin, Q. Chen, and S. Yan, "Network in network," 2013, *arXiv:1312.4400*.
- [92] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. Alemi, "Inception-v4, inception-resnet and the impact of residual connections on learning," in *Proc. AAAI Conf. Artif. Intell.*, 2017, pp. 4278–4284.
- [93] Z. Jia et al., "Beyond data and model parallelism for deep neural networks," in *Proc. Mach. Learn. Syst.*, 2019, pp. 1–13.
- [94] M.-T. Luong, H. Pham, and C. D. Manning, "Effective approaches to attention-based neural machine translation," 2015, *arXiv:1508.04025*.
- [95] H. Zhu et al., "Daydream: Accurately estimating the efficacy of optimizations for DNN training," in *Proc. USENIX Annu. Tech. Conf.*, 2020, pp. 337–352.
- [96] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [97] Y. Wu et al., "Google's neural machine translation system: Bridging the gap between human and machine translation," 2016, *arXiv:1609.08144*.
- [98] H. Hu et al., "dPRO: A generic performance diagnosis and optimization toolkit for expediting distributed DNN training," in *Proc. Mach. Learn. Syst.*, 2022, pp. 623–637.
- [99] G. Lu et al., "DistSim: A performance model of large-scale hybrid distributed DNN training," 2023, *arXiv:2306.08423*.
- [100] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, 2019, Art. no. 9.

- [101] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, pp. 1–67, 2020.
- [102] J. Duan et al., "Proteus: Simulating the performance of distributed DNN training," 2023, *arXiv:2306.02267*.
- [103] M. Naumov et al., "Deep learning recommendation model for personalization and recommendation systems," 2019, *arXiv: 1906.00091*.
- [104] S. Zhang, X. Yi, L. Diao, C. Wu, S. Wang, and W. Lin, "Expediting distributed DNN training with device topology-aware graph deployment," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 4, pp. 1281–1293, Apr. 2023.
- [105] S. Chib et al., "Understanding the metropolis-hastings algorithm," *Amer. Statistician*, vol. 49, pp. 327–335, 1995.
- [106] C. NVIDIA, Profiling tools interface (CUPTI). [Online]. Available: <https://developer.nvidia.com/cuda-profiling-tools-interface>
- [107] A. Jayarajan, J. Wei, G. Gibson, A. Fedorova, and G. Pekhimenko, "Priority-based parameter propagation for distributed DNN training," in *Proc. Mach. Learn. Syst.*, vol. 1, pp. 132–145, 2019.
- [108] NVIDIA, API documentation of NVIDIA's apex optimizers, 2018.
- [109] A. B. Kahn, "Topological sorting of large networks," *Commun. ACM*, vol. 5, pp. 558–562, 1962.
- [110] R. W. Hockney, "The communication challenge for MPP: Intel paragon and Meiko CS-2," *Elsevier Parallel Comput.*, vol. 20, pp. 389–398, 1994.
- [111] J. Zhou et al., "Graph neural networks: A review of methods and applications," *Elsevier AI Open*, vol. 1, pp. 57–81, 2020.
- [112] L. Kocsis et al., "Bandit based Monte-Carlo planning," in *Proc. Springer Eur. Conf. Mach. Learn.*, 2006, pp. 282–293.
- [113] P. Xie et al., "Distributed machine learning via sufficient factor broadcasting," 2015, *arXiv:1511.08486*.
- [114] S.-H. Lin, M. Paolieri, C. -F. Chou, and L. Golubchik, "A model-based approach to streamlining distributed training for asynchronous SGD," in *Proc. IEEE 26th Int. Symp. Model. Anal. Simul. Comput. Telecommun. Syst.*, 2018, pp. 306–318.
- [115] M. Reiser, "A queueing network analysis of computer communication networks with window flow control," *IEEE Trans. Commun.*, vol. 27, no. 8, pp. 1199–1209, Aug. 1979.
- [116] Z. Li et al., "Throughput prediction of asynchronous SGD in TensorFlow," in *Proc. ACM/SPEC Int. Conf. Perform. Eng.*, 2020, pp. 76–87.
- [117] C. Yang et al., "PerFestimator: A generic and extensible performance estimator for data parallel DNN training," in *Proc. 2021 IEEE/ACM Int. Workshop Cloud Intell.*, 2021, pp. 13–18.
- [118] H. Casanova et al., "Versatile, scalable, and accurate simulation of distributed applications and platforms," *Elsevier JPDC*, vol. 74, pp. 2899–2917, 2014.
- [119] J. Wang et al., "DNNEmu: A lightweight performance emulator for distributed DNN training," in *Proc. Springer Int. Conf. Algorithms Archit. Parallel Process.*, 2022, pp. 722–736.
- [120] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 3149–3157.
- [121] T. Chen et al., "MXNet: A flexible and efficient machine learning library for heterogeneous distributed systems," 2015, *arXiv:1512.01274*.
- [122] D. Justus, J. Brennan, S. Bonner, and A. S. McGough, "Predicting the computational cost of deep learning models," in *Proc. 2018 IEEE Int. Conf. Big Data*, 2018, pp. 3873–3882.
- [123] X. Y. Geoffrey et al., "Habitat: A runtime-based computational performance predictor for deep neural network training," in *Proc. 2021 USENIX Annu. Techn. Conf.*, 2021, pp. 503–521.
- [124] Y. Gao, X. Gu, H. Zhang, H. Lin, and M. Yang, "Runtime performance prediction for deep learning models with graph neural network," in *Proc. 2023 IEEE/ACM 45th Int. Conf. Softw. Eng. Softw. Eng. Pract.*, 2023, pp. 368–380.
- [125] K. Santhanam et al., "DistIR: An intermediate representation for optimizing distributed neural networks," in *Proc. Workshop Mach. Learn. Syst.*, 2021, pp. 15–23.
- [126] C. Lattner et al., "MLIR: A compiler infrastructure for the end of Moore's law," 2020, *arXiv: 2002.11054*.
- [127] A. Samajdar et al., "SCALE-sim: Systolic CNN accelerator simulator," 2018, *arXiv: 1811.02883*.
- [128] N. Agarwal, T. Krishna, L. -S. Peh, and N. K. Jha, "GARNET: A detailed on-chip network model inside a full-system simulator," in *Proc. 2009 IEEE Int. Symp. Perform. Anal. Syst. Softw.*, 2009, pp. 33–42.
- [129] T. R. Henderson et al., "Network simulations with the NS-3 simulator," *SIGCOMM Demonstration*, vol. 14, 2008, Art. no. 527.
- [130] S. Sridharan et al., "Chakra: Advancing performance benchmarking and co-design using standardized execution traces," 2023, *arXiv:2305.14516*.
- [131] H. Wang et al., "Efficient and systematic partitioning of large and deep neural networks for parallelization," in *Proc. 27th Int. Conf. Parallel Distrib. Comput.*, Lisbon, Portugal, Springer, Sep. 1–3, 2021, pp. 201–216.
- [132] D. K. Kadiyala, S. Rashidi, T. Heo, A. R. Bambhaniya, T. Krishna, and A. Daglis, "COMET: A comprehensive cluster design methodology for distributed deep learning training," 2022, *arXiv:2211.16648*.
- [133] A. Abdi, S. Rashidi, F. Fekri, and T. Krishna, "Restructuring, pruning, and adjustment of deep models for parallel distributed inference," 2020, *arXiv: 2008.08289*.
- [134] O. Terzo et al., *Heterogeneous Computing Architectures: Challenges and Vision*. Boca Raton, FL, USA: CRC Press, 2019.
- [135] Q. Zeng, Y. Du, K. Huang, and K. K. Leung, "Energy-efficient resource management for federated edge learning with CPU-GPU heterogeneous computing," *IEEE Trans. Wireless Commun.*, vol. 20, no. 12, pp. 7947–7962, Dec. 2021.
- [136] G. Lim et al., "Zico: Efficient GPU memory sharing for concurrent DNN training," in *Proc. USENIX Annu. Techn. Conf.*, 2021, pp. 161–175.
- [137] A. Bakhoda, G. L. Yuan, W. W. L. Fung, H. Wong, and T. M. Aamodt, "Analyzing CUDA workloads using a detailed GPU simulator," in *Proc. 2009 IEEE Int. Symp. Perform. Anal. Syst. Softw.*, 2009, pp. 163–174.
- [138] B. M. Beckmann and A. Gutierrez, "The AMD gem5 APU simulator: Modeling heterogeneous systems in gem5," in *Proc. Tut. Int. Symp. Microarchit.*, 2015.
- [139] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, "DRAMsim3: A cycle-accurate, thermal-capable DRAM simulator," *IEEE Comput. Archit. Lett.*, vol. 19, no. 2, pp. 106–109, Jul./Dec. 2020.
- [140] X. Chang, "Network simulations with opnet," in *Proc. 31st Conf. Winter Simul.*, 1999, pp. 307–314.
- [141] P. Mattson et al., "MLPerf: An industry standard benchmark suite for machine learning performance," *IEEE Micro*, vol. 40, no. 2, pp. 8–16, Mar./Apr. 2020.
- [142] Q. Yang et al., "DeepQueueNet: Towards scalable and generalized network performance estimation with packet-level visibility," in *Proc. ACM SIGCOMM Conf.*, 2022, pp. 441–457.
- [143] K. Shen and B. Li, "Learning-based network performance estimators: The next frontier for network simulation," *IEEE Net.*, vol. 37, no. 4, pp. 97–103, Jul./Aug. 2023.
- [144] Y.-C. Liao, C. -C. Wang, C. -H. Tu, M. -C. Kao, W. -Y. Liang, and S. -H. Hung, "PerfNetRT: Platform-aware performance modeling for optimized deep neural networks," in *Proc. IEEE 2020 Int. Comput. Symp.*, 2020, pp. 153–158.
- [145] Y. Peng, Y. Bao, Y. Chen, C. Wu, C. Meng, and W. Lin, "DL2: A deep learning-driven scheduler for deep learning clusters," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 8, pp. 1947–1960, Aug. 2021.
- [146] Y. Gao et al., "SpotLight: Optimizing device placement for training deep neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1676–1684.



Zhenhua Guo received the PhD degree from the College of Computer Science and Technology, Harbin Engineering University, China, in 2016. His research interest includes computer system architecture and heterogeneous computing.



Yinan Tang received the PhD degree from the Beijing University of Posts and Telecommunications, China, in 2020. His research interest includes computer system architecture and deep learning system.



Jidong Zhai (Senior Member, IEEE) received the BS degree in computer science from the University of Electronic Science and Technology of China, Chengdu, China, in 2003, and the PhD degree in computer science from Tsinghua University, Beijing, China, in 2010. He is a professor with the Department of Computer Science and Technology, Tsinghua University. His research interests include performance evaluation for high performance computers, performance analysis, and modeling of parallel applications.



Li Wang received the MS degree from the Shanghai Institute of Microsystems and Information Technology, the University of Chinese Academy of Sciences, China, in 2015. Her research interest includes heterogeneous computing and artificial intelligence.



Tongtong Yuan received the PhD degree from the Beijing University of Posts and Telecommunications, Beijing, China, in 2020. She is currently a faculty of Information Technology, Beijing University of Technology, Beijing, China. Her research interests include computer vision, deep learning, computer networks.



Yaqian Zhao received the PhD degree from the School of Computer Science and Engineering, Beihang University, China, in 2014. Her research interests include computer system architecture and artificial intelligence.



Jian Jin is currently working with Industrial Internet and IOT Research Institute, China Academy of Information and Communications Technology, Beijing, China.



Rengang Li received the MS degree from the College of Computer Science and Technology, the National University of Defense Technology, China, in 2008. His current research interests include computer system architecture, heterogeneous computing, and artificial intelligence.